

Mini Project Design Document

Online Order Management System API using FastAPI and PostgreSQL

1. Project Title

Mini Online Order Management System

This project is a simple e-commerce backend API similar to an online shopping platform. Customers can view products, add products to a cart, place orders, and view order details.

2. Project Objective

The main objective of this mini project is to learn how to build a REST API using **FastAPI** and store data using **PostgreSQL**.

By completing this project, students will learn:

- FastAPI project structure
 - PostgreSQL database connection
 - SQLAlchemy ORM
 - Pydantic schemas
 - CRUD API development
 - Table relationships
 - Foreign keys
 - Request and response validation
 - Error handling
 - Swagger UI API testing
-

3. Technology Stack

Area	Technology
Programming Language	Python
Backend Framework	FastAPI
Database	PostgreSQL
ORM	SQLAlchemy
Data Validation	Pydantic
API Testing	Swagger UI / Postman

Area	Technology
Database Tool	pgAdmin 4

4. Project Scope

This is a beginner-friendly mini project. The system will include only the main features needed for an online order system.

Included Features

- Customer management
- Product management
- Cart management
- Order placement
- Order item management
- Simple payment status
- Product stock update
- Order status update

Not Included in First Version

- Login and authentication
- Real online payment gateway
- Product image upload
- Delivery tracking
- Seller management
- Discount coupons
- Product reviews

These can be added later as advanced features.

5. Main Modules

The project contains the following modules:

1. Customer Module
 2. Product Module
 3. Cart Module
 4. Order Module
 5. Payment Module
-

6. Module Details

6.1 Customer Module

This module manages customer details.

Customer Data

Field	Description
id	Unique customer ID
name	Customer name
email	Customer email
phone	Customer phone number
address	Customer address
created_at	Created date and time

Customer APIs

Method	Endpoint	Description
POST	<code>/customers</code>	Create a new customer
GET	<code>/customers</code>	Get all customers
GET	<code>/customers/{customer_id}</code>	Get customer by ID
PUT	<code>/customers/{customer_id}</code>	Update customer
DELETE	<code>/customers/{customer_id}</code>	Delete customer

6.2 Product Module

This module manages products available in the online store.

Product Data

Field	Description
id	Unique product ID
name	Product name
description	Product description
price	Product price
stock_quantity	Available stock
category	Product category

Field	Description
created_at	Created date and time

Product APIs

Method	Endpoint	Description
POST	/products	Create a new product
GET	/products	Get all products
GET	/products/{product_id}	Get product by ID
PUT	/products/{product_id}	Update product
DELETE	/products/{product_id}	Delete product

Optional Query Parameters

Endpoint	Example	Description
/products?category=mobile	Filter by category	
/products?search=phone	Search product by name	
/products?min_price=1000&max_price=5000	Filter by price range	

6.3 Cart Module

This module allows customers to add products to a shopping cart before placing an order.

Cart Item Data

Field	Description
id	Unique cart item ID
customer_id	Customer ID
product_id	Product ID
quantity	Product quantity
created_at	Created date and time

Cart APIs

Method	Endpoint	Description
POST	/cart	Add product to cart
GET	/customers/{customer_id}/cart	View customer cart

Method	Endpoint	Description
PUT	<code>/cart/{cart_item_id}</code>	Update cart item quantity
DELETE	<code>/cart/{cart_item_id}</code>	Remove item from cart

6.4 Order Module

This module manages customer orders.

When a customer places an order, the system should:

1. Get all cart items of the customer.
2. Calculate the total amount.
3. Create an order.
4. Create order items.
5. Reduce product stock.
6. Clear the customer cart.

Order Data

Field	Description
id	Unique order ID
customer_id	Customer ID
total_amount	Total order amount
status	Order status
created_at	Created date and time

Order Status Values

Pending
Confirmed
Packed
Shipped
Delivered
Cancelled

Order APIs

Method	Endpoint	Description
POST	<code>/orders/customer/{customer_id}</code>	Place order for customer
GET	<code>/orders</code>	Get all orders

Method	Endpoint	Description
GET	<code>/orders/{order_id}</code>	Get order by ID
GET	<code>/customers/{customer_id}/orders</code>	Get orders by customer
PUT	<code>/orders/{order_id}/status</code>	Update order status

6.5 Order Items Module

Each order can have many products. These products are stored as order items.

Order Item Data

Field	Description
id	Unique order item ID
order_id	Order ID
product_id	Product ID
quantity	Ordered quantity
unit_price	Product price at order time
subtotal	quantity × unit_price

6.6 Payment Module

This module stores simple payment details. No real payment gateway is required.

Payment Data

Field	Description
id	Unique payment ID
order_id	Order ID
payment_method	Payment method
payment_status	Payment status
paid_amount	Paid amount
created_at	Created date and time

Payment Methods

Cash on Delivery
Card
Bank Transfer

Payment Status Values

Pending
Paid
Failed
Refunded

Payment APIs

Method	Endpoint	Description
POST	<code>/payments</code>	Add payment record
GET	<code>/payments/{payment_id}</code>	Get payment by ID
GET	<code>/orders/{order_id}/payment</code>	Get payment by order
PUT	<code>/payments/{payment_id}/status</code>	Update payment status

7. Database Design

7.1 Tables

The system should contain the following PostgreSQL tables:

1. customers
2. products
3. cart_items
4. orders
5. order_items
6. payments

7.2 Table Relationships

Customer 1 ---- many Cart Items
Customer 1 ---- many Orders

Product 1 ---- many Cart Items
Product 1 ---- many Order Items

Order 1 ---- many Order Items
Order 1 ---- one Payment

7.3 Database Table Structure

customers

Column	Type	Constraint
id	Integer	Primary Key
name	String	Not Null
email	String	Unique, Not Null
phone	String	Not Null
address	String	Not Null
created_at	DateTime	Default Current Time

products

Column	Type	Constraint
id	Integer	Primary Key
name	String	Not Null
description	String	Nullable
price	Numeric	Not Null
stock_quantity	Integer	Not Null
category	String	Not Null
created_at	DateTime	Default Current Time

cart_items

Column	Type	Constraint
id	Integer	Primary Key
customer_id	Integer	Foreign Key
product_id	Integer	Foreign Key
quantity	Integer	Not Null

Column	Type	Constraint
created_at	DateTime	Default Current Time

orders

Column	Type	Constraint
id	Integer	Primary Key
customer_id	Integer	Foreign Key
total_amount	Numeric	Not Null
status	String	Default Pending
created_at	DateTime	Default Current Time

order_items

Column	Type	Constraint
id	Integer	Primary Key
order_id	Integer	Foreign Key
product_id	Integer	Foreign Key
quantity	Integer	Not Null
unit_price	Numeric	Not Null
subtotal	Numeric	Not Null

payments

Column	Type	Constraint
id	Integer	Primary Key
order_id	Integer	Foreign Key, Unique
payment_method	String	Not Null
payment_status	String	Default Pending
paid_amount	Numeric	Not Null
created_at	DateTime	Default Current Time

8. Business Rules

8.1 Product Rules

- Product price must be greater than 0.
- Product stock quantity cannot be negative.
- Product name is required.

8.2 Cart Rules

- Customer must exist before adding items to cart.
- Product must exist before adding to cart.
- Quantity must be greater than 0.
- Customer cannot add more quantity than available stock.
- If the same product already exists in cart, update the quantity instead of creating a duplicate row.

8.3 Order Rules

- Customer must have at least one cart item before placing an order.
- Order total should be calculated using cart item quantity and product price.
- After placing an order, product stock should be reduced.
- After placing an order, customer cart should be cleared.
- Cancelled orders should not be changed to Delivered.

8.4 Payment Rules

- Payment amount should match the order total.
 - One order should have only one payment record.
 - Payment status should be Pending by default.
-

9. Suggested Project Folder Structure

```
online_order_api/  
|  
├─ main.py  
├─ database.py  
├─ models.py  
├─ schemas.py  
├─ crud.py  
├─ requirements.txt  
└─ routers/  
    ├─ customers.py  
    ├─ products.py  
    └─ cart.py
```

```
|— orders.py
|— payments.py
```

10. API Testing Flow

Students should test the system in this order:

1. Create a customer.
2. Create products.
3. View product list.
4. Add products to customer cart.
5. View customer cart.
6. Place an order.
7. Check order details.
8. Check product stock quantity.
9. Add payment record.
10. Update order status.

11. Sample Test Scenario

Step 1: Add Customer

Customer:

```
{
  "name": "Amal Perera",
  "email": "amal@gmail.com",
  "phone": "0771234567",
  "address": "Kandy"
}
```

Step 2: Add Product

Product:

```
{
  "name": "Wireless Mouse",
  "description": "Bluetooth wireless mouse",
  "price": 2500,
  "stock_quantity": 20,
  "category": "Electronics"
}
```

Step 3: Add Product to Cart

Cart item:

```
{
  "customer_id": 1,
  "product_id": 1,
  "quantity": 2
}
```

Step 4: Place Order

Endpoint:

```
POST /orders/customer/1
```

Expected result:

```
{
  "customer_id": 1,
  "total_amount": 5000,
  "status": "Pending"
}
```

12. Student Tasks

Basic Tasks

Students must complete:

- Create FastAPI project
- Connect PostgreSQL database
- Create SQLAlchemy models
- Create Pydantic schemas
- Create customer CRUD APIs
- Create product CRUD APIs
- Create cart APIs
- Create order placement API
- Create payment APIs
- Test all APIs using Swagger UI

Advanced Tasks

Students can try:

- Product search
 - Product category filter
 - Low stock product list
 - Customer order history
 - Order cancellation
 - Best-selling products report
 - Daily sales total API
-

13. Final Output Expected

At the end of the project, students should submit:

1. Source code
 2. PostgreSQL database tables
 3. Swagger UI screenshots
 4. API testing screenshots
 5. Short explanation of the project
 6. Final demo
-

14. Evaluation Criteria

Criteria	Marks
Project setup and database connection	10
Database table design	15
Customer and product CRUD APIs	20
Cart functionality	15
Order placement logic	20
Payment and order status handling	10
Code quality and folder structure	5
Final demo and explanation	5
Total	100

15. Learning Outcome

After completing this mini project, students will be able to build a real-world backend API using FastAPI and PostgreSQL. They will understand how to design database tables, create API endpoints, validate request data, handle relationships, and implement basic business logic for an online ordering system.
